



# Trabajo Práctico Especial Arquitectura de Computadoras

Grupo 4

Integrantes:

Juan Ichazo 66236

Federico Gomboc 66363

Martina María Fernandez Binetti 66861

---

3 de Junio, 2026

## Introducción

El presente trabajo describe el diseño, la implementación y la depuración de un Sistema Operativo de 64 bits desarrollado a partir de una arquitectura base provista por la cátedra. El objetivo principal del proyecto fue construir un entorno de ejecución funcional que integre tanto el modo privilegiado (Kernel) como el espacio de usuario (Userland), gestionando correctamente la interacción entre el hardware y el software.

A lo largo del desarrollo, implementamos los controladores esenciales del sistema (teclado, video, sonido y temporizador), un conjunto de syscalls para la comunicación entre el kernel y userland, y una interfaz de línea de comandos (Shell). Sobre esto logramos ejecutar exitosamente programas de usuario complejos, que van desde herramientas de cálculo intensivo (como bruteforce) hasta un entorno gráfico interactivo para el juego Pacman.

## Manejo de Excepciones y Captura de Estado

Uno de los aspectos más importantes de un sistema operativo es su capacidad para mantener el control, manejar errores imprevistos y reaccionar a eventos inesperados de manera segura. Por eso, queremos abordar cómo implementamos el manejo de excepciones y la captura del estado de los registros.

Uno de los primeros desafíos técnicos que tuvimos fue extraer los registros justo en el momento que se aprieta una tecla (nosotros elegimos tab) sin que se corrompan. Para realizarlo, optamos por guardarlos en un arreglo constantemente de forma preventiva y cuando el keyboard driver lee un tab, toma el arreglo para después imprimirlo. Si guardáramos los registros después de leer el tab, es probable que los registros se ensucien en el proceso y se impriman los valores equivocados.

Por otro lado, nos dimos cuenta de que al guardar los registros del stack, estábamos utilizando un registro como auxiliar pero que no estábamos preservando. Lo solucionamos pusheando el registro a la pila previo a su uso y luego popeando cuando terminaba de utilizarlo. Sin embargo, como se utilizaba para extraer registros de la pila, el puntero a todos estos registros ahora tenía un defasaje de 8 bytes más, que debíamos tener en cuenta.

Este mismo código para capturar registros resultó útil para el manejo de excepciones ya que se pidió que, además de recuperarse y retornar a la shell, se debe imprimir el estado de los registros al momento de la excepción. Sin embargo, no podíamos convertir ese código en una función ya que al llamarla se modificarían la pila y los registros. Por este motivo, optamos por hacer que sea una macro, que evita este problema.

Por último, el manejo de excepciones se implementó cuidadosamente tomando en cuenta los registros que se debían modificar y los que no, para luego pushearlos en el orden correcto para que iretq los popee. Primero que nada se hizo la snapshot de los registros, luego se obtuvo el id de la excepción y dicho arreglo con los registros. De este último extrajimos los registros CS, RFLAGS y SS, que son los que no se modificaron. Luego, se llamó a la función getStackBase para obtener el inicio de la pila y se lo asignamos a RSP. A RIP se le asignó la dirección de inicio de userland. El orden en el que se pushearons los

registros en la pila es el siguiente: SS, RSP, RFLAGS, CS, RIP, de forma que iretq los reciba en el orden inverso.

## Drivers

Para el teclado se implementó un buffer que almacena las teclas a medida que se ingresan por medio de la interrupción del teclado. Se ofrece una syscall que levanta estas teclas almacenadas en el buffer y las devuelve al usuario. Este driver también se encarga de las teclas especiales como tab, alt y ctrl (ambas reproducen un sonido).

Se utilizó el driver de video provisto por la cátedra y se agregaron funciones que permiten dibujar rectángulos, o directamente dibujar desde una matriz de píxeles. También se implementó una función que permite el scroll de la consola.

A la hora de implementar el driver de sonido, el problema principal a afrontar fue que usar llamadas como sleep para reproducir melodías frenaba la ejecución del programa. Para evitar esto se implementó una función que guarda una melodía y actualiza el sonido usando la interrupción del reloj interno de la computadora. De esta manera se pudieron proveer dos syscalls, una que frena la ejecución y otra que reproduce la melodía en el fondo.

## Implementación Pacman

Uno de los principales desafíos para la implementación del juego fue la lógica de la función que mueve los fantasmas (en modo 1 jugador). Nuestro objetivo fue que cada fantasma cumpla con su personalidad original del juego, persiguiendo al pacman de distintas formas. Blinky, el fantasma rojo, persigue directamente al pacman; Pinky, el rosa, apunta 4 tiles adelante del pacman; Inky, el celeste, trata de acorralar al pacman utilizando a Blinky; y Clyde, el naranja, persigue al pacman pero cuando está a menos de 8 tiles escapa. Al llegar a una intersección, la IA evalúa las celdas adyacentes válidas y calcula la distancia hacia el objetivo. El fantasma siempre elige la dirección que minimice esta distancia, y se incorporó la restricción de que los fantasmas no pueden revertir su dirección de movimiento de forma instantánea.

En cuanto al diseño del juego, el mayor reto gráfico fue evitar el parpadeo visual. Nos dimos cuenta que dibujar la pantalla entera en cada iteración del game loop resultaba muy costoso, por lo que implementamos un sistema que rastrea qué regiones de la pantalla habían cambiado. Mediante una matriz de estado, el sistema marca exclusivamente los tiles donde las entidades estaban en el frame anterior y donde se encuentran ahora. Durante el ciclo de renderizado, la pantalla solo actualiza y restaura el fondo de esos bloques específicos antes de dibujar a los personajes por encima.

## Syscalls

Decidimos implementar el syscall dispatcher en el Kernel utilizando un arreglo de punteros a funciones, donde el índice del arreglo corresponde al ID de la llamada. Esto nos permite despachar las peticiones de manera eficiente y con un orden de complejidad temporal  $O(1)$ . Al recibir una interrupción de software, el Kernel verifica que el ID solicitado no exceda el tamaño del arreglo de syscalls. Si un programa en Userland intenta ejecutar una llamada con un ID inválido, la solicitud es rechazada de forma segura, previniendo accesos a direcciones de memoria incorrectas.

Finalmente, construimos una biblioteca estándar (libc) en Userland. Esta biblioteca proporciona wrappers para las syscalls, y es gracias a esto que los programas de Userland pueden consumir los servicios del hardware a través de una API. Además, proporciona funciones para imprimir o leer por salida estándar como `printf` y `scanf`.

## Kernel Cracker

Para optimizar la función de hash, se decidió que lo más eficiente sería escribirla directamente en assembler para evitar todas las precauciones que tiene el compilador gcc. Para lograr esto, se compiló la función aparte usando el *compiler explorer* con flags de optimización (O3 y Ofast). De esta manera, se redujo ampliamente la cantidad de instrucciones de la función.

Para medir el tiempo transcurrido se usó la directiva `rdtsc` para obtener la cantidad de ciclos del CPU que ocurrieron. Luego se mide la cantidad de ciclos en 1 ms para obtener el tiempo transcurrido en milisegundos. Se definió que el algoritmo de fuerza bruta intentase con todos los strings de tamaño menor a una variable definida previamente (actualmente 5).

## Conclusión

A lo largo de este proyecto logramos desarrollar un sistema operativo funcional de 64 bits que integra los componentes fundamentales de un entorno de ejecución. Partiendo de una arquitectura base, implementamos mecanismos de manejo de excepciones, captura de registros, drivers de dispositivos, syscalls, una biblioteca estándar para Userland y una Shell capaz de ejecutar distintos programas de usuario.

El trabajo nos permitió comprender en profundidad la interacción entre hardware y software, así como la importancia de preservar correctamente el estado del procesador, administrar interrupciones y diseñar interfaces seguras entre el Kernel y las aplicaciones.

En conclusión, este proyecto nos permitió poner en práctica muchos de los conceptos vistos, desde programación en Assembler hasta el manejo de interrupciones, excepciones y la comunicación entre Kernel y Userland. También nos enfrentó a problemas reales de diseño, implementación y depuración que requirieron comprender en detalle el funcionamiento interno del sistema. Como resultado, logramos construir un sistema operativo capaz de ejecutar distintos programas de usuario e integrar correctamente todos los componentes desarrollados a lo largo del trabajo.